YOLO26 Model Training and Results

Chair and table detector (2 classes) trained on the chair-table dataset

Student	Aleti Vivek Reddy
Student ID	415410902
Course	UAV (CCU) - YOLO26 hands-on, class of November 3
Model	YOLO26n (yolo26n.pt) fine-tuned for 2 classes: 0 = chair, 1 = table
Code	github.com/vvvvvivekkk/UAV (train.py, predict_mine.py, make_report.py)

Contents

0. Setup and dataset
1. Training loss curves
2. Validation loss curves
3. Precision, Recall and mAP
4. Detection results using my trained model
5. Conclusion

0. Setup and dataset

Item	Value
Software	Python 3.13.17, Ultralytics 8.4.208, PyTorch 2.15.0+cpu
Hardware	CPU only (no GPU)
Dataset	chair-table: 150 labelled room photos, YOLO .txt format
Classes	0 = chair (142 boxes), 1 = table (119 boxes)
Split	120 train (212 boxes) / 30 val (49 boxes), 0 backgrounds, 0 corrupt
Base model	yolo26n.pt (pretrained on COCO), fine-tuned
Settings (args.yaml)	epochs=50, imgsz=640, batch=16, optimizer=auto, seed=0

data.yaml: `train: images/train  val: images/val  names: {0: chair, 1: table}`. Training command: `model.train(data="chair-table/data.yaml", epochs=50, imgsz=640)`.

```
TERMINAL OUTPUT - python train.py (start, epoch 1, epoch 50, final validation)

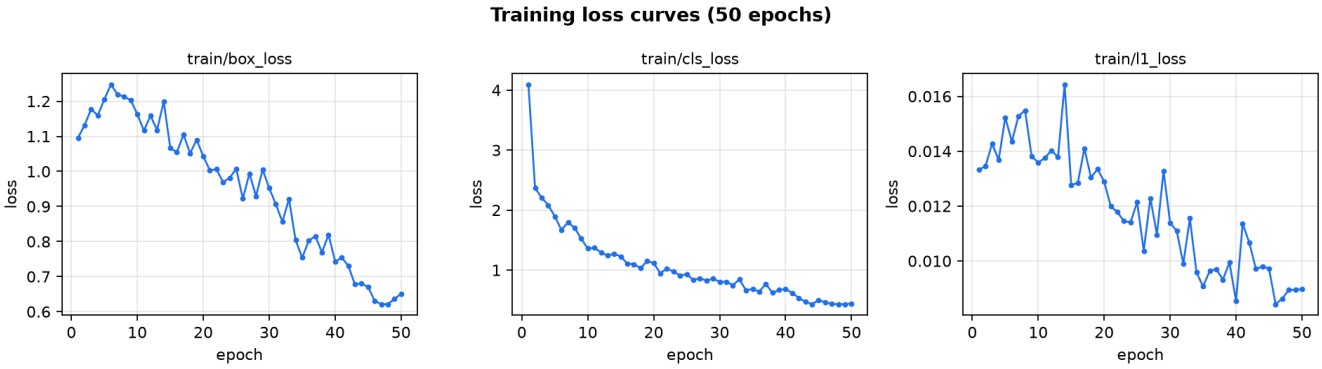
$ python train.py
train: Scanning chair-table/labels/train... 120 images, 0 backgrounds, 0 corrupt:
val: Scanning chair-table/labels/val... 30 images, 0 backgrounds, 0 corrupt:
Starting training for 50 epochs...

      Epoch    GPU_mem   box_loss   cls_loss    l1_loss  Instances       Size
       1/50         0G      1.096      4.093    0.01332         28        640:
                  all         30         49      0.493      0.604      0.603      0.392
  ...
      50/50         0G     0.6508     0.4422   0.008972         11        640:
                  all         30         49      0.819      0.732      0.791      0.535

50 epochs completed in 0.956 hours.
Optimizer stripped from runs/detect/train/weights/last.pt, 5.5MB
Optimizer stripped from runs/detect/train/weights/best.pt, 5.5MB
Validating runs/detect/train/weights/best.pt...
Ultralytics 8.4.208  Python-3.13.17 torch-2.15.0+cpu CPU (Intel Xeon Processor @ 2.10GHz)
YOLO26n summary (fused): 121 layers, 2,389,978 parameters, 0 gradients, 5.4 GFLOPs
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95):
                   all         30         49      0.862      0.705      0.817      0.562
                 chair         22         27      0.948      0.682      0.844      0.605
                 table         19         22      0.775      0.727       0.79      0.518
Speed: 1.9ms preprocess, 97.7ms inference, 0.0ms loss, 0.3ms postprocess per image
Results saved to runs/detect/train
EXIT 0
```

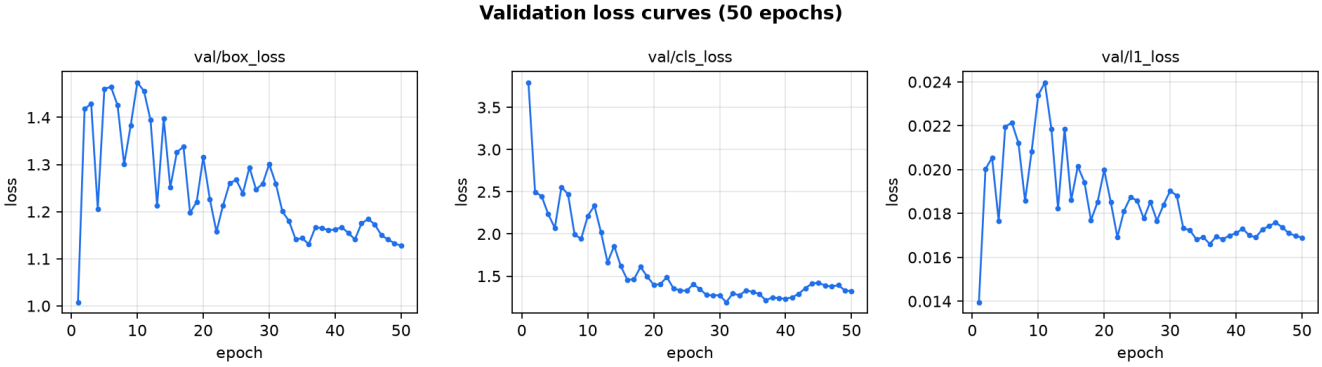
Terminal output of the training run, taken from the saved log (train_log.txt).

1. Training loss curves



All three training losses go down over the 50 epochs. **box_loss** (how far the predicted boxes are from the labelled boxes) fell from 1.096 to 0.651; **cls_loss** (how wrong the chair/table class guess is) fell from 4.093 to 0.442; **l1_loss** fell from 0.0133 to 0.0090. YOLO26 uses an L1 box loss in place of the Distribution Focal Loss (DFL) used in earlier YOLO versions, so there is no dfl_loss column.

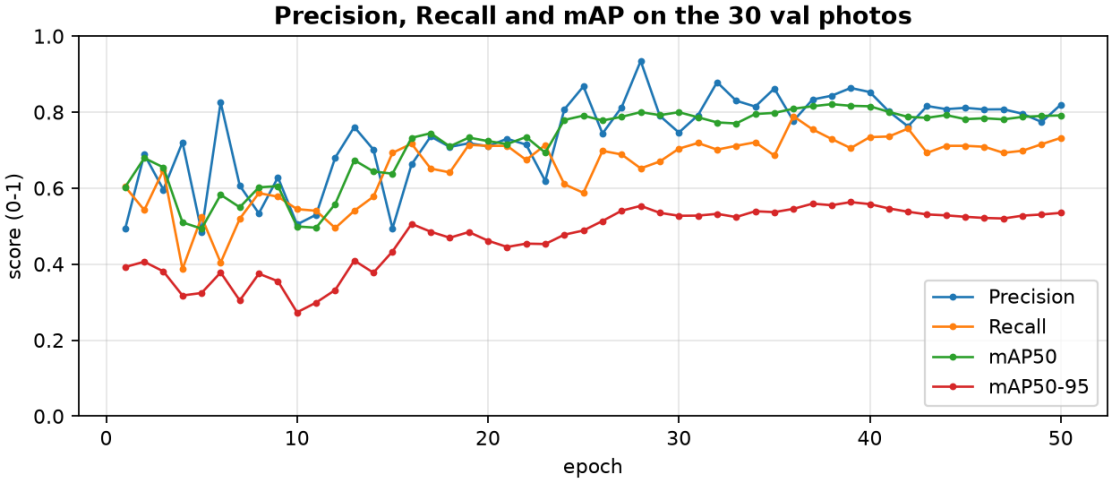
2. Validation loss curves



Validation losses are measured on the 30 val photos, which the model never trains on. val/box_loss went from 1.008 to 1.128, val/cls_loss from 3.794 to 1.322 and val/l1_loss from 0.0140 to 0.0169. The first ~10 epochs jump up and down while the new 2-class head adapts; afterwards the curves settle. Validation loss does not rise at the end, so there is no clear sign of overfitting in 50 epochs.

3. Precision, Recall and mAP

Precision: of the boxes the model drew, the share that are correct. **Recall**: of the real chairs and tables, the share the model found. **mAP50**: mean average precision when a box counts as correct if it overlaps the label by at least 50% (IoU 0.5). **mAP50-95**: the same averaged over stricter overlaps from 0.50 to 0.95.



	Precision	Recall	mAP50	mAP50-95
Epoch 1	0.493	0.604	0.603	0.392
Epoch 50 (last.pt)	0.819	0.732	0.791	0.535
Best epoch 39	0.863	0.705	0.817	0.563

Final validation of best.pt, per class

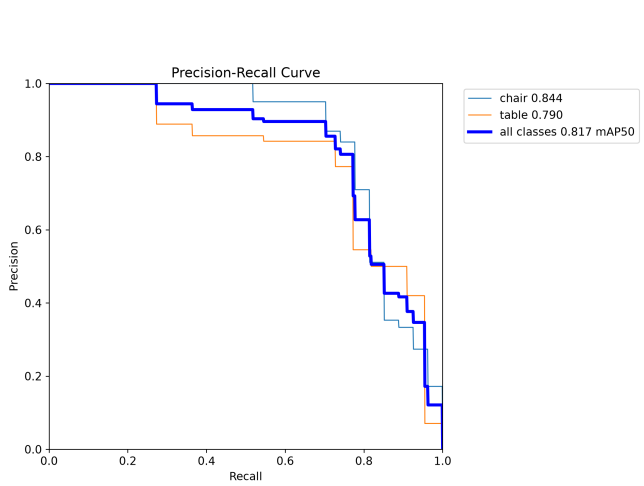
Class	Boxes	Precision	Recall	mAP50	mAP50-95
all	49	0.864	0.723	0.815	0.564
chair	27	0.951	0.718	0.842	0.605
table	22	0.776	0.727	0.788	0.523

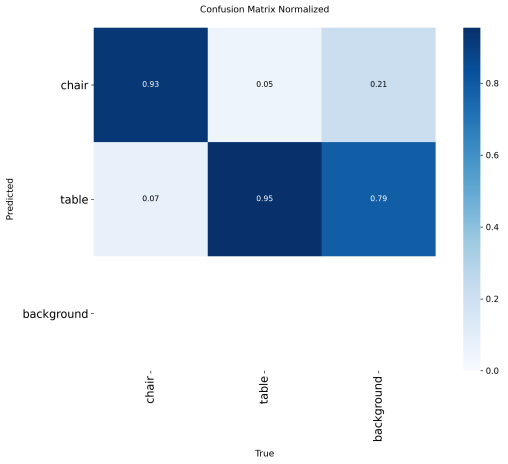
The per-class table comes from re-validating best.pt with model.val(). It differs by about 0.002 from the end-of-training validation shown on page 2 (mAP50 0.817), because the two runs use slightly different validation batching.

```
TERMINAL OUTPUT - validation of best.pt on the 30 val photos

$ python make_report.py   # best.pt .val(data='chair-table/data.yaml')
Ultralytics 8.4.208  Python-3.13.17 torch-2.15.0+cpu
             Class     Images  Instances      Box(P          R      mAP50  mAP50-95)
               all         30         49      0.864      0.723      0.815      0.564
             chair                    27      0.951      0.718      0.842      0.605
             table                    22      0.776      0.727      0.788      0.523
Speed: 3.4ms preprocess, 91.7ms inference, 0.2ms postprocess per image
```

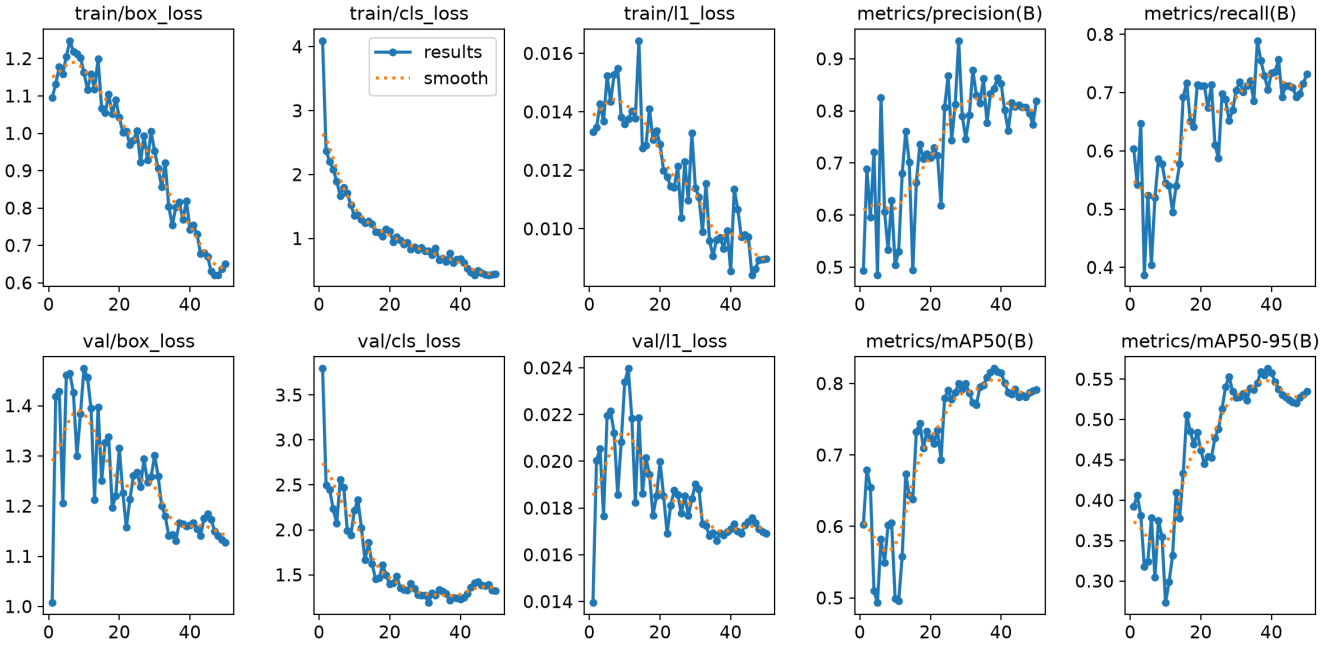
3b. PR curve and confusion matrix (from the training run)





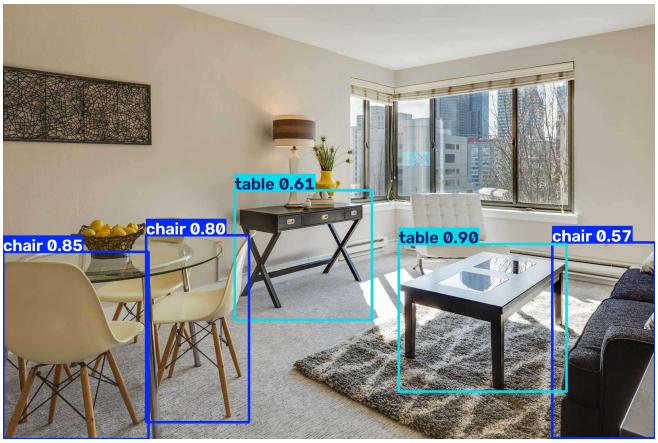
Left: precision-recall curve per class (area under it = AP). Right: confusion matrix, showing how often each true class was predicted as chair, table, or missed (background).

Ultralytics summary chart (results.png)

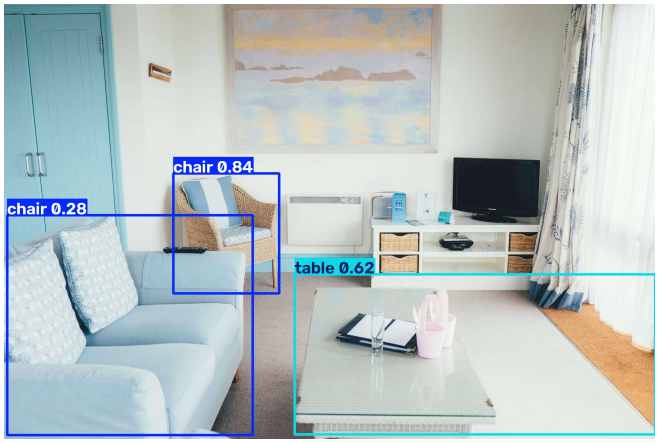


4. Detection results using my trained model

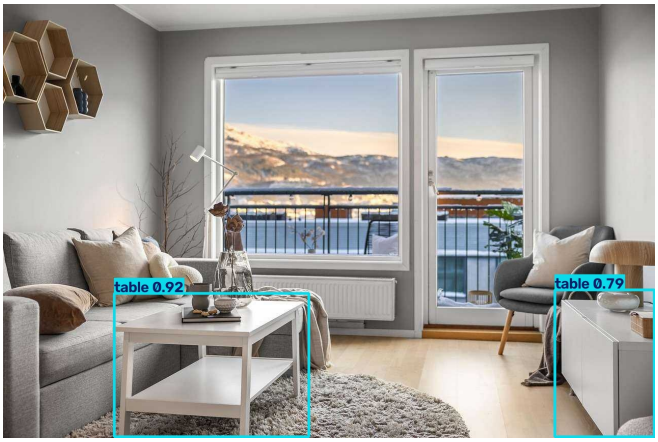
best.pt was run on validation photos it never trained on (conf = 0.25). Each caption lists the boxes found, compared with the number of labelled objects.



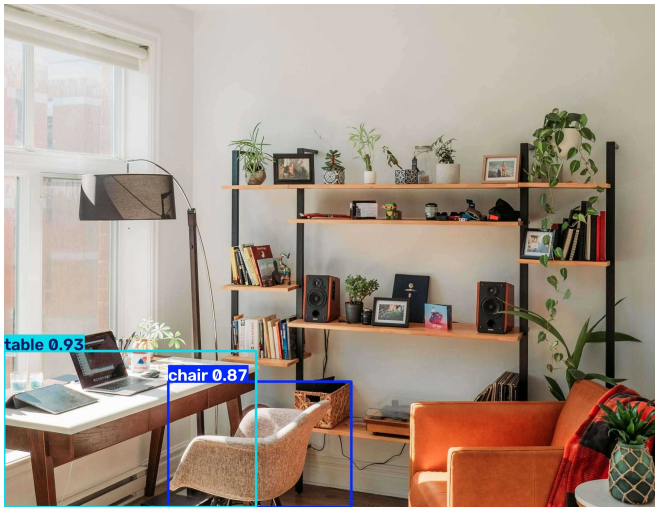
val/138.jpg - found 3 chairs, 2 tables (labelled: 2 chairs, 3 tables)
table 0.90, chair 0.85, chair 0.80, table 0.61, chair 0.57



val/144.jpg - found 2 chairs, 1 tables (labelled: 1 chairs, 1 tables)
chair 0.84, table 0.62, chair 0.28



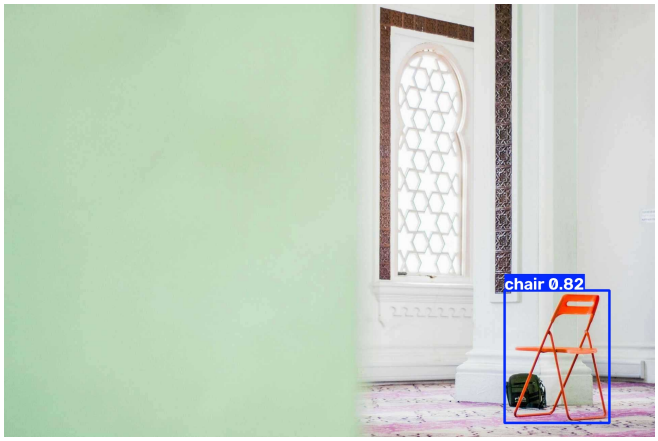
val/130.jpg - found 0 chairs, 2 tables (labelled: 1 chairs, 1 tables)
table 0.92, table 0.79



val/140.jpg - found 1 chairs, 1 tables (labelled: 1 chairs, 1 tables)
table 0.93, chair 0.87



val/149.jpg - found 0 chairs, 3 tables (labelled: 0 chairs, 1 tables)
table 0.91, table 0.39, table 0.35



val/121.jpg - found 1 chairs, 0 tables (labelled: 1 chairs, 0 tables)
chair 0.82

TERMINAL OUTPUT - detection with my trained best.pt

```
$ python make_report.py   # runs best.pt .predict(conf=0.25) on 6 val photos
image chair-table/images/val/138.jpg: 3 chairs, 2 tables
image chair-table/images/val/144.jpg: 2 chairs, 1 tables
image chair-table/images/val/130.jpg: 0 chairs, 2 tables
image chair-table/images/val/140.jpg: 1 chairs, 1 tables
image chair-table/images/val/149.jpg: 0 chairs, 3 tables
image chair-table/images/val/121.jpg: 1 chairs, 0 tables
Results saved to runs/detect/report_predict
```

Validation batch: my labels vs. my model's boxes



Left: labelled boxes (val_batch0_labels.jpg). Right: predictions of the trained model (val_batch0_pred.jpg).

5. Conclusion

I fine-tuned YOLO26n for two classes (chair, table) for 50 epochs on CPU. Training and validation losses both decreased, and on the 30 unseen val photos the best model reaches precision 0.864, recall 0.723, mAP50 0.815 and mAP50-95 0.564 (chair mAP50 0.842, table mAP50 0.788). The detection images show it finds dining chairs, armchairs, desks and coffee tables in rooms it never saw.

Limitations and next steps: only 120 training photos and 30 val photos, so scores move a lot between epochs; small or partly hidden chairs are sometimes missed or boxed twice. More photos (especially of my own rooms), more epochs on a GPU, and a larger model (yolo26s) would improve accuracy.

All numbers come from runs/detect/train/results.csv and model.val() on runs/detect/train/weights/best.pt. Scripts and AI-assistant use: the training and report scripts were written with Claude (AI coding assistant) and run by me.